Kotlin Coroutines

Kotlin coroutines provide a structured, lightweight mechanism for asynchronous, non-blocking programming. Rather than blocking OS threads during long-running operations (such as network I/O or disk access), coroutines *suspend* execution at specific points, freeing the underlying thread to execute other tasks and resuming seamlessly when the result is ready.

1. Coroutines Basics & Structured Concurrency 2

Core Builders & Scope Functions 2

Suspending Functions 3

2. Cancellation & Timeouts 3

Making Code Cancellable 3

Cleanup & Non-Cancellable Blocks 4

Timeouts 4

3. Composing Suspending Functions 5

Sequential vs. Concurrent Execution 5

Lazy async 5

4. Coroutine Context & Dispatchers 5

Standard Dispatchers 6

Context Combination & Switching 6

5. Asynchronous Flow & Operators 7

Core Characteristics of Cold Flows 7

Flow Operators Reference 7

6. Channels (Hot Communication Primitives) 9

Channel Types & Capacities 9

Channel Patterns 10

7. Exception Handling & Supervision 10

Supervision (SupervisorJob & supervisorScope) 10

CoroutineExceptionHandler 10

Cancellation vs. Exceptions 11

8. Shared Mutable State & Concurrency 11

1. Coroutines Basics & Structured Concurrency

Coroutines always execute within a **CoroutineScope**, which enforces **structured concurrency**:

- **Scope Lifetime:** New coroutines can only be launched within a delimited CoroutineScope. The outer scope will not complete until all of its child coroutines have finished.
- **Automatic Cancellation:** If a scope or parent coroutine is cancelled or fails with an unhandled exception, all of its children are automatically cancelled, preventing resource and memory leaks.

Core Builders & Scope Functions

Builder / Function	Behavior	Return Type	Threading Impact
runBlocking	Bridges regular blocking code and suspending code.	T (block result)	**Blocks** the current thread until all inner coroutines complete.
launch	Fire-and-forget builder for tasks that do not return a value.	Job	**Non-blocking**; inherits context from its parent CoroutineScope.
async	Starts a concurrent computation that returns a single future result.	Deferred<T>	**Non-blocking**; call .await() on the Deferred to suspend for the result.
coroutineScope	Creates a child scope and waits for all launched children to finish.	T (block result)	**Suspends** (does not block) the current thread; cancels all children if any child fails.
supervisorScope	Like coroutineScope, but isolates failures downward.	T (block result)	**Suspends**; failure of one child does **not** cancel siblings or the scope itself.

Suspending Functions

Functions marked with the suspend modifier can pause execution of a coroutine without blocking the thread. They can only be invoked from another suspending function or within a coroutine builder.

```
import kotlinx.coroutines.*

fun main() =
  runBlocking { // Bridges blocking main() to suspending world
    doWorld()
    println("Done")
  }

suspend fun doWorld() =
  coroutineScope { // Suspends until both launches finish
    launch {
        delay(1000L) // Non-blocking suspension for 1 second
        println("World 1")
    }
    launch {
        delay(2000L)
        println("World 2")
    }
    println("Hello")
  }

// Output: Hello -> World 1 -> World 2 -> Done
```

2. Cancellation & Timeouts

Coroutine cancellation is **cooperative**. Calling job.cancel() transitions the coroutine to a *Cancelling* state, but if the coroutine is executing a tight CPU-bound loop without checking for cancellation, it will continue running until it finishes.

Making Code Cancellable

1. **Use kotlinx.coroutines suspending functions:** Functions like delay(), yield(), and withContext() automatically check for cancellation and throw a CancellationException when cancelled.
2. **Check isActive or call ensureActive():** Inside CPU-heavy loops, explicitly verify the coroutine state on CoroutineScope.

```
val job = launch(Dispatchers.Default) {
    var i = 0
    while (isActive) { // Cooperative check (or call
ensureActive())
        // Perform CPU-intensive calculation
        i++
    }
}
delay(500L)
job.cancelAndJoin() // Cancels the job and suspends until
completion
```

Cleanup & Non-Cancellable Blocks

When a coroutine is cancelled, it throws CancellationException. Standard try { ... } finally { ... } blocks execute normally to release resources. However, any attempt to call a suspending function inside the finally block will immediately throw another CancellationException unless wrapped in withContext(NonCancellable):

```
val job = launch {
    try {
        repeat(1000) { i ->
            delay(500L)
        }
    } finally {
        withContext(NonCancellable) {
            delay(100L) // Safe to suspend inside finally
            println("Cleanup completed safely")
        }
    }
}
```

Timeouts

- **withTimeout(timeMillis):** Cancels the running block and throws TimeoutCancellationException (a subclass of CancellationException) if the deadline is exceeded.
- **withTimeoutOrNull(timeMillis):** Returns null instead of throwing an exception upon timeout.

3. Composing Suspending Functions

By default, code inside a coroutine is **sequential**. Calling two suspending functions one after another executes the second only after the first finishes.

Sequential vs. Concurrent Execution

To run independent suspending operations concurrently, wrap them in async blocks inside a coroutineScope:

```
suspend fun fetchUserDashboard(): Dashboard = coroutineScope {
    // Both requests start concurrently
    val profileDeferred = async { fetchUserProfile() }
    val postsDeferred = async { fetchUserPosts() }

    // Suspends until both results are available; if either fails,
    //   the other is cancelled
    Dashboard(profileDeferred.await(), postsDeferred.await())
}
```

Lazy async

Passing start = CoroutineStart.LAZY to async delays execution until .await() or .start() is explicitly called. Note: If you call .await() on two lazy async blocks sequentially without calling .start() first, they will execute sequentially rather than concurrently.

4. Coroutine Context & Dispatchers

Every coroutine executes within a **CoroutineContext**—an immutable, indexed set of elements (Element instances). The primary elements are:

- **Job:** Controls the lifecycle and parent-child hierarchy of the coroutine.
- **CoroutineDispatcher:** Determines which thread or thread pool the coroutine runs on.
- **CoroutineName:** Assigns a human-readable name for debugging.
- **CoroutineExceptionHandler:** Handles uncaught exceptions in root coroutines.

Standard Dispatchers

Dispatcher	Backing Thread Pool	Ideal Use Case
Dispatchers.Default	Shared pool of background threads (sized to CPU core count, min 2).	CPU-intensive work (sorting large lists, JSON parsing, complex math).
Dispatchers.IO	Elastic pool of on-demand threads (up to 64 threads or core count).	Blocking I/O operations (network requests, file/disk reads, database queries).
Dispatchers.Main	Single UI thread (Android Main, JavaFX, Swing).	UI updates and lightweight immediate state coordination.
Dispatchers.Unconfined	Starts in caller thread; resumes in whatever thread the suspending function used.	Specific edge cases where thread confinement is unneeded and dispatch overhead must be avoided.

Context Combination & Switching

- **Combining Elements (+ operator):** Combine multiple context elements when building a scope or launching a coroutine:

```
launch(Dispatchers.Default + CoroutineName("Worker")) { ... }
```

- **Switching Contexts (withContext):** Changes the context (usually the dispatcher) for a specific block of code while remaining in the same coroutine:

```
suspend fun loadAndRenderData() { // Runs on Dispatchers.Main
 val data = withContext(Dispatchers.IO) {
  networkApi.fetchData() // Performs IO off the main thread
 }
 updateUi(data) // Automatically switches back to main thread
}
```

- **ThreadLocal Propagation:** Because coroutines bounce between threads as they suspend and resume, standard ThreadLocal variables lose their values unless bound to the coroutine context via myThreadLocal.asContextElement(value = "scopedValue").

5. Asynchronous Flow & Operators

While a suspending function returns a single value asynchronously, a **Flow<T>** returns multiple asynchronously computed values sequentially.

Core Characteristics of Cold Flows

- **Cold Streams:** Code inside a flow { ... } builder does not run until a terminal operator (such as .collect()) is called. Each new collector triggers an independent execution of the flow.
- **Context Preservation:** A flow always collects in the context of the calling coroutine. You cannot emit values from a different CoroutineContext inside a standard flow { ... } block.
- **flowOn Operator:** The official way to change the upstream context of a flow (without affecting downstream operators or .collect()):

```
fun numbersFlow(): Flow<Int> = flow {
    for (i in 1..3) {
        delay(100)
        emit(i) // Executes on Dispatchers.Default
    }
}.flowOn(Dispatchers.Default)
```

Flow Operators Reference

Category	Operator	Behavior
Intermediate (Transform)	map, filter, take, drop	Transforms or filters upstream emissions synchronously or via suspending lambdas.
General Transform	transform	Emits arbitrary values any number of times per upstream item (can simulate map, filter, etc.).
Buffering / Conflation	buffer()	Runs upstream producer concurrently with a slow downstream collector using a channel buffer.

	conflate()	Drops intermediate values if the collector is too slow, processing only the most recent emission.
	collectLatest { ... }	Cancels and restarts the collector block every time a new value is emitted upstream.
Combining Multiple Flows	zip	Pairs emissions 1-to-1 from two flows; waits for both to emit a new item before producing a pair.
	combine	Re-computes whenever *either* flow emits, pairing the latest value from each flow.
Flattening (Flow<Flow<T>>)	flatMapConcat	Waits for each inner flow to complete before starting to collect the next one sequentially.
	flatMapMerge	Collects incoming inner flows concurrently and merges their emissions as they arrive.
	flatMapLatest	Cancels collection of the current inner flow immediately when a new outer value is emitted.
Exception & Completion	catch { e -> ... }	Catches *upstream* exceptions only (transparent to downstream exceptions). Can emit() fallback data.

	onCompletion { cause -> ... }	Executes like a finally block after the flow completes normally (cause == null) or exceptionally.
Terminal Operators	collect, toList, first, single, reduce, fold	Suspends the calling coroutine and triggers execution of the cold flow pipeline.
Non-Blocking Launch	launchIn(scope)	Shorthand for scope.launch { flow.collect() }; returns a Job without suspending the caller.

6. Channels (Hot Communication Primitives)

While Flow is a cold stream of data, a **Channel<T>** is a **hot** synchronization primitive conceptually similar to a non-blocking BlockingQueue. Instead of blocking threads, it uses suspending send(value) and receive() operations to pass data between coroutines.

```
val channel = Channel<Int>()
launch {
    for (x in 1..5) channel.send(x * x)
    channel.close() // Signals that no more items will be sent
}
for (y in channel) println(y) // Receives until channel is closed
```

Channel Types & Capacities

When creating a Channel<T>(capacity), the capacity parameter determines how send() behaves when no receiver is ready:

1. **Channel.RENDEZVOUS (0 - Default):** Unbuffered. send() suspends until another coroutine invokes receive(), and vice versa.
2. **Channel.BUFFERED (64 by default) or Fixed Size (N):** Buffers up to N elements before send() suspends.
3. **Channel.CONFLATED:** Retains only the single most recently sent element, overwriting any unread element; send() never suspends.
4. **Channel.UNLIMITED:** Backed by an unbounded linked list (Int.MAX_VALUE); send() never suspends, though memory exhaustion (OutOfMemoryError) is possible if consumers fall behind.

Channel Patterns

- **Producer-Consumer (produce):** The produce { send(x) } coroutine builder launches a coroutine that feeds a channel and returns a ReceiveChannel<T>, automatically closing the channel when the coroutine finishes.
- **Fan-Out:** Multiple worker coroutines read from the *same* channel to distribute work across concurrent consumers. Use a regular for (msg in channel) loop inside each worker safely.
- **Fan-In:** Multiple producer coroutines send elements to the *same* shared channel.
- **Ticker Channels:** Produces Unit at fixed time intervals (note: largely superseded by Flow time operators in modern idioms).

7. Exception Handling & Supervision

Coroutine builders fall into two propagation models for uncaught exceptions:

1. **Propagating automatically (launch):** Treats exceptions as unhandled failures. The exception immediately cancels the parent Job, which cancels all sibling coroutines and propagates up the hierarchy.
2. **Exposing to the user (async / produce):** Encapsulates exceptions inside the resulting Deferred or Channel object, rethrowing them when .await() or .receive() is called. *(Warning: If an async coroutine is a direct child of a regular Job or coroutineScope, it will still immediately cancel its parent upon failure even before .await() is called!)*

Supervision (SupervisorJob & supervisorScope)

To prevent a failing child coroutine from bringing down its parent and siblings (crucial for UI components or server request handlers), use supervision:

- **SupervisorJob():** Propagates cancellation only *downward*. If a child launched inside CoroutineScope(SupervisorJob()) fails, siblings keep running.
- **supervisorScope { ... }:** A scoped block where children fail independently. Inside a supervisorScope, exceptions thrown inside async blocks are **not** propagated until .await() is explicitly called, allowing standard try/catch around .await().

CoroutineExceptionHandler

A last-resort global catch block for logging or crash reporting:

- Only works when installed on a **root coroutine** (a direct child of CoroutineScope) or a direct child of a **SupervisorJob**.
- Has no effect on async (since async expects the caller to catch exceptions during .await()).
- Does **not** prevent cancellation; by the time it runs, the coroutine has already completed exceptionally.

Cancellation vs. Exceptions

- CancellationException is ignored by the coroutine machinery as a normal mechanism to terminate a coroutine; it cancels children but **never cancels the parent**.
- If a coroutine encounters multiple exceptions while cancelling its children, the first exception wins, and any subsequent exceptions are attached via Throwable.addSuppressed().

8. Shared Mutable State & Concurrency

Because coroutines using multi-threaded dispatchers (Dispatchers.Default or Dispatchers.IO) can execute in parallel across multiple CPU cores, accessing shared mutable variables requires explicit synchronization.

Synchronization Strategy	Mechanism	When to Use
1. Mutex (Mutual Exclusion)	val mutex = Mutex() mutex.withLock { counter++ }	**Best general-purpose choice.** Suspends waiting coroutines without blocking threads when protecting critical sections.
2. Atomic Primitives	AtomicInteger, AtomicReference (java.util.concurrent.atomic / kotlinx.atomicfu)	Fast, lock-free thread safety for single primitive counters or reference updates; insufficient for complex multi-variable state invariants.
3. Thread Confinement (Fine-Grained)	val singleThread = Dispatchers.Default.limitedParallelism(1) withContext(singleThread) { state++ }	Confines all mutations of a state object to a single sequential dispatcher queue; switches contexts on every access.
4. Thread Confinement (Coarse-Grained)	Run the entire business logic coroutine on a single-threaded dispatcher (e.g., Dispatchers.Main).	Ideal for UI state managers where all reads and writes naturally live on one thread without context switching overhead.

Why @Volatile is not enough: Marking a property @Volatile guarantees linearizable reads and writes (visibility across threads), but does **not** provide atomicity for compound actions like counter++ (which reads, increments, and writes in three separate steps).

```
import kotlinx.coroutines.*
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock

val mutex = Mutex()
var sharedCounter = 0

suspend fun safeIncrement() = coroutineScope {
  repeat(1000) {
    launch(Dispatchers.Default) {
      mutex.withLock { // Doesn't block OS threads
        sharedCounter++
      }
    }
  }
}
```